Reviewer Pack — Minimal Order of Geometric Quantization and SAT Clause Set S…

Entry c2528146623d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 07:01:34 UTC

Minimal Order of Geometric Quantization and SAT Clause Set Simplicity Inequality

Entry ID: `c2528146623d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 07:01:34 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization **Field B** (complexity object): Boolean Satisfiability (SAT) Clause Sets

Statement:

> For every instance of size n, the minimal order of geometric quantization invariants for a polynomial representing the clause set of an SAT formula is O(n^2)

Rationale (proposer's reasoning):

> Geometric quantization can provide a bridge between algebraic geometry and quantum information theory. This conjecture proposes that the minimal order of geometric quantization invariants for a polynomial representing a SAT clause set can be used to bound the simplicity of the clause set, which is a measure of how many variables are involved in any clause.

Taxonomy category: `Geometric_Quantization_to_SAT` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `21bc61832039868e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the seeds (n=30) yield a minimal order of geometric quantization invariants that is O(n^2), measured as |order - n^2| ≤ 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `geometric quantization AND minimal order AND Boolean satisfiability` - `SAT clause sets AND geometric quantization AND order O(n^2)` - `quantization invariants AND size n AND Boolean Satisfiability`

Top relevant hits considered: - [http://arxiv.org/abs/1908.01624v1] Learned Clause Minimization in Parallel SAT Solvers - [http://arxiv.org/abs/2207.13577v2] Scalable Proof Producing Multi-Threaded SAT Solving with Gimsatul through Sharing instead of Copying Clauses - [http://arxiv.org/abs/1505.03340v2] HordeSat: A Massively Parallel Portfolio SAT Solver

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            clauses.append(clause)
        return clauses

    def polynomial_from_cnf(cnf):
        n = len(cnf[0])
        poly = [[0] * n for _ in range(n)]
        for clause in cnf:
            for literal in clause:
                var = abs(literal) - 1
                if literal > 0:
                    poly[var][var] += 1
                else:
                    poly[var][var] -= 1
        return poly

    def minimal_order_of_invariants(poly):
        n = len(poly)
        order = 0
        for i in range(n):
            for j in range(i + 1, n):
```

```
                if poly[i][j] != 0:
                    order += 1
        return order

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        cnf = generate_cnf(n)
        poly = polynomial_from_cnf(cnf)
        order = minimal_order_of_invariants(poly)

        if order > n * (n - 1) // 2:
            return {
                "metric_name": "minimal_order_of_invariants",
                "metric_value": order,
                "instances_tested": 1,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": f"order > {n * (n - 1) // 2}"
            }

        results.append({
            "metric_name": "minimal_order_of_invariants",
            "metric_value": order,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": abs(order - n * (n - 1) // 2) <= 3
        })

    return results[0]

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"order > {n * (n - 1) // 2}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
variants', 'metric_value': 0, 'instances_tested': 1, 'n_max': 5, 'conjecture_holds': False}
TRIAL: {'metric_name': 'minimal_order_of_invariants', 'metric_value': 0, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'minimal_order_of_invariants', 'metric_value': 0, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'minimal_order_of_invariants', 'metric_value': 0, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'minimal_order_of_invariants', 'metric_value': 0, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'minimal_order_of_invariants', 'metric_value': 0, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'minimal_order_of_invariants', 'metric_value': 0, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'minimal_order_of_invariants', 'metric_value': 0, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'minimal_order_of_invariants', 'metric_value': 0, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'minimal_order_of_invariants', 'metric_value': 0, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'minimal_order_of_invariants', 'metric_value': 0, 'instances_tested': 1, 'n_m

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74445b4a.py", line 99, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74445b4a.py", line 99, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                                   ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Review and debug the test code to ensure it can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12044
2	preregistration	ollama_remote	glm4:latest	0	0	9434
3	novelty	ollama_remote	glm4:latest	0	0	7996
4	novelty	ollama_remote	glm4:latest	0	0	9605
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27802
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9459

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8638
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10481
9	judge	ollama_remote	glm4:latest	0	0	26912

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 122372 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c2528146623d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c2528146623d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c2528146623d.tar.gz` (if generated)